

Once you are done building the package, you should be able to check it for errors with the help of Lintian, which checks for common packaging errors/warnings and reports any policy violations if present. The above tutorials is obviously not the most comprehensive way to learn packaging in Linux, rather it serves to provide you with a skeletal introduction to get you started in more detail.

Red Hat/Fedora based packaging

The other popular package management system is the Red Hat package Manager which is used in most of the Red Hat based systems. Obviously, Red Hat is much more popular among enterprise users and organizations because of its enterprise support availability as opposed to Debian, which is mostly used by independent developers and is community supported.

RPM packages are provided in the form of rpm files and has the following comparable functions: -

- Installing a deb package:
`rpm -i packagename.rpm`
- Removing an installed package:
`rpm -e packagename`
- List of installed packages:
`rpm -qa`
- Query whether a package is installed or not:
`rpm -qa | grep packagename`
- Determine the package name to which a given file belongs to:
`rpm -qf /path/to/file`
- Show metadata about a package:
`rpm -ql packagename`

As you might have guessed rpm command is used to deal with low level rpm files, however yum can be used to deal with higher level functions like dependency resolution etc. as follows: -

- Install and update software on the system:
`yum install packagename`
`yum update packagename`
- Remove software:
`yum erase packagename`
- Update package list:
`yum check-update`
- Upgrade the system:
`yum update`
- Search for a package:
`yum list packagename`

Packaging software for the red hat based systems

Similar to debian packaging, you will need the following packages to start building rpm files: -

```
yum install @development-tools
yum install fedora-packager
```

Now, to start building the package, you will need to create a directory structure according to the defined guidelines under the software's subdirectory. These are mainly the following directories: -

- SPECS : This is where the specification files are stored.
- SOURCES : The source directory where the sources tarballs and patches are supposed to be kept.
- BUILD : Source packages unpacked and compiled here.
- BUILDROOT : Files are installed under this directory.
- RPMS : Binary rpms are created under this directory.
- SRPMS : Source rpms are created here.

So basically all you need to do is create a .spec file under the SPECS directory and place the tarball into SOURCES directory, then you can run the rpmbuild command from the SPECS directory as follows: -

```
rpmbuild -ba filename.spec
```

Obviously, the spec file here is the most important file that provides most of the packaging information, and an empty file with an example of its contents would look as follows: -

```
Name:
Version:
Release:    1%{?dist}
Summary:
Group:
License:
URL:
Source0:
BuildRoot:  %{_tmppath}/%{name}-%{version}-
%{release}-root-%(%{_id_u} -n)
BuildRequires:
Requires:
%description
%prep
%setup -q
%build
%configure
make %{?_smp_mflags}
%install
rm -rf %{buildroot}
make install DESTDIR=%{buildroot}
%clean
rm -rf %{buildroot}
%files
%defattr(-,root,root,-)
%doc
%changelog
```

As you can see, this file provides all the information pertaining to each stage in the rpmbuild process. Once the binary rpm is ready, you can check the rpm for errors with the rpmlint utility as follows: -

```
rpmlint filename.spec ../RPMS/packagename.rpm ../
SRPMS/packagename.rpm
```

Creating binary packages for the software is only part of the process. Usually, maintaining your packages is part of a much larger, continuous process. While some people prefer to use Debian based systems and others use Red Hat based ones, the developer who is publishing software for Linux has to ensure that all their bases are covered.

There are a lot of other package management systems used (often interchangeably) in other distributions like zypp in openSUSE, urpmi in Mandriva/Mageia, pacman in Arch, etc. Targeting for all of these is obviously not possible, but once you have packages available for the above two and your software is accepted into the repositories of these popular Linux distributions, it can get you a lot of visibility and eventually it may automatically be adopted by other users and maintainers working in these lesser used platforms.

Apart from creating packages for your own application, you can also choose to adopt some orphaned packages that have been put up for adoption as they might need a maintainer. This will give you a lot of technical know-how and open a window into how open source development works. >